

Your Agent Is a Distributed System

Debugging AI Failures with OpenTelemetry and OpenSearch

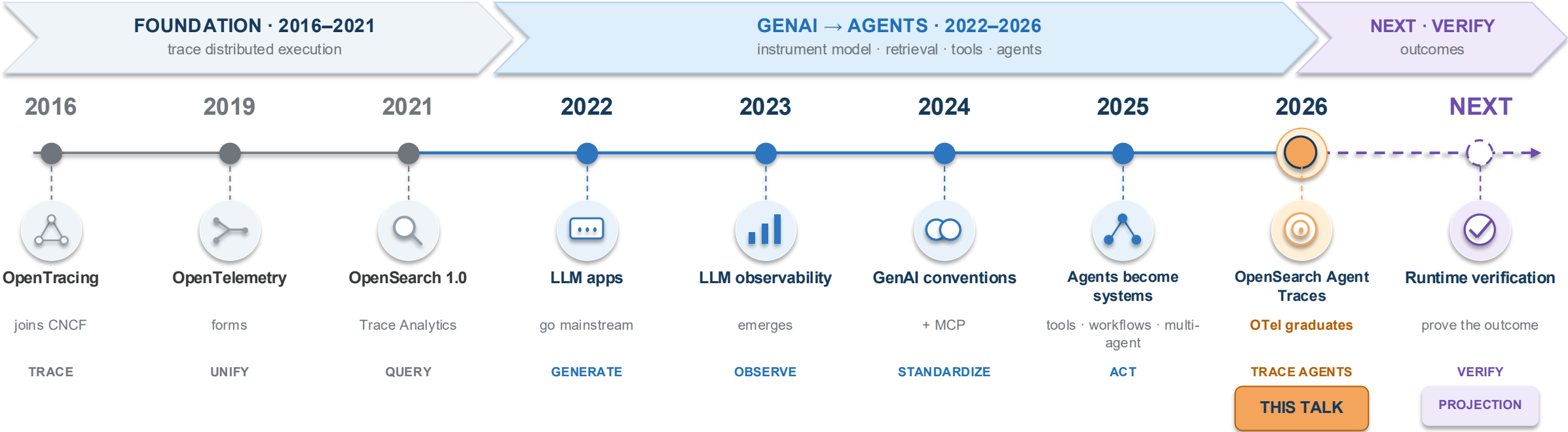
Harishankar Menderkar · Sarat Chandra

Analytics, Security & Observability · San Jose



From tracing to verification — how we got here

Tracing standardized how we follow execution. Agentic systems add the next requirement: prove the outcome



An agent that only answers needs watching. An agent that acts needs proof that the action worked

Sources: CNCF · OpenTelemetry · OpenSearch Project · Anthropic · OpenAI | NEXT = projection



A national TV & streaming platform. A very bad Sunday night.

The platform

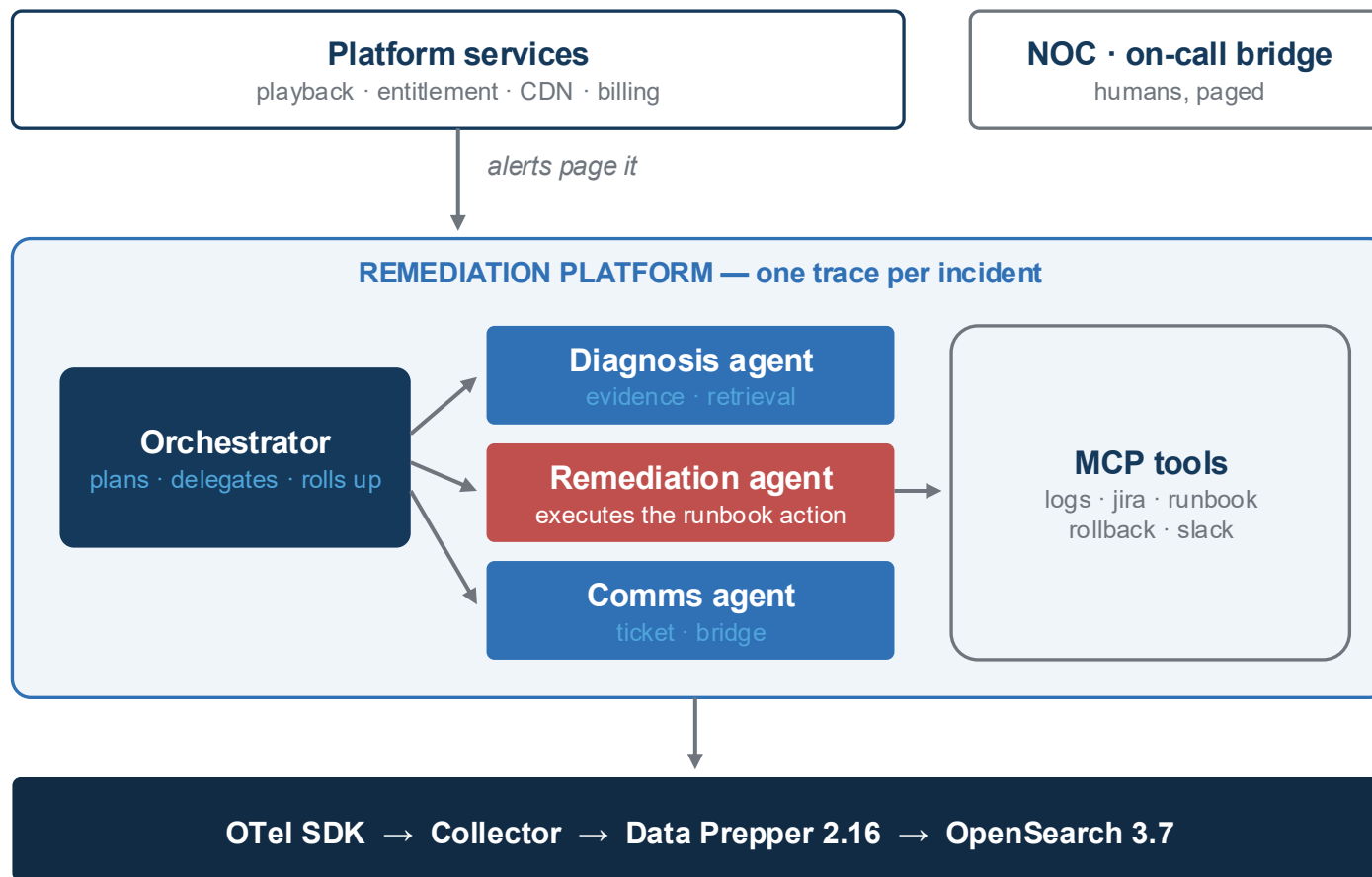
Tens of millions of subscribers. Live sports and entertainment. The worst minutes are live-event windows — kickoff traffic hits playback, entitlement, CDN, and billing at once.

The operations reality

Hundreds of alerts in a bad window. The NOC cannot open every one. Time-to-first-action decides how many subscribers notice.

The platform's answer

Incidents page a **multi-agent remediation platform** before a human joins the bridge: an orchestrator delegates to specialist agents that diagnose, act, and record.



every incident = one distributed trace across four agent services



You added an AI agent. Now something is wrong.

An incident-remediation copilot — paged during a live-event window, it gathers evidence over MCP, executes the approved runbook action, and updates the ticket.

“It's slowest exactly when we need it.”

“Dashboards are green but answers are slow.”

“It answered with a platitude during a real outage.”

— filed as “the agent is sometimes wrong”, which is not a debuggable statement.

Logs record **what happened**. Traces record **why it took that path**.

An agent is a plan step, a retrieval, N tool calls, a generation. That is a distributed system — and we already know how to debug those.

```
INFO agent started
INFO tool called
INFO tool returned
INFO tool called
INFO tool returned
INFO llm responded
INFO agent done
```

↑ what you have. Where is the bug?



The platform, instrumented — 4 agent services, 1 trace per incident

diagnosis-agent — opensearch_logs · jira_mcp — evidence

remediation-agent — runbook_mcp · rollback_mcp — ACTS

comms-agent — jira_mcp · slack_mcp — record, notify

orchestrator — plans, delegates, rolls up the run

Every root span carries **agent.synthetic=true** · seed 42 · fully deterministic

Dual attributes on every LLM span:

`gen_ai.*` for Agent Traces classification

`llm.*` stable fallbacks — conventions still in Development

```
incident.remediate      remediation-orchestrator
├─ llm.plan              plan · delegate
├─ agent.run diagnosis   diagnosis-agent
│   ├── retrieval.search  doc_count · bucket
│   ├── tool.call logs · jira  + tool.attempt each
│   ├── context.assemble   tokens · overflow
│   └─ llm.generate + usage_check ← custom span
├─ agent.run remediation remediation-agent
│   └─ tool.call rollback_mcp attempt=1 timeout X / 2 ✓
└─ agent.run comms        comms-agent
    └─ tool.call jira · slack record + notify
```

Fleet triage is one root-span query (`00_triage.ppl`):

500 incidents → **104 retried** · **95 heavy retrieval** · **51 evidence-lost**

Detection reads emitted signals only — never the synthetic scenario label.



Traces don't fix your agent. They tell you what to fix.

The method — same four steps every time

- 1 Symptom** — an unactionable complaint: “it's slow”, “it's sometimes wrong”
- 2 Signal** — one attribute the agent already emits — an attempt number, a document bucket, a ratio
- 3 Query** — one PPL statement over the whole fleet: how often, how bad, which boundary
- 4 Fix** — a human changes code or config — now aimed at the right thing

Most teams are stuck at step 1 because step 2 was never recorded.

EXAMPLE

```
symptom  "the rollback is slow during events"
signal    tool.retry.attempt = 2  on tool.attempt
query     104 of 500 incidents — all rollback_mcp
fix       raise timeout · backoff · idempotency key
```

This platform is the X-ray, not the surgeon.

It finds the fault, proves how often it happens, and ranks what to fix first. Engineers still write the fix — but they stop guessing.

Every slide that follows is one pass of this loop



The remediation agent executes the rollback. The logical call **succeeds**

Attempt 1 times out. Attempt 2 succeeds. The application log records one success — no exception raised. For a tool that writes, a silent retry is a double-execution risk

Explicit `tool.retry.attempt` on each attempt span makes this directly queryable — no inference from duplicate span names

104 of 500 incidents —
20.8%

the ROLLBACK retried silently · every log line says “success”
FIX raise the MCP client timeout, add jittered backoff and idempotency keys, alert on retry rate — the attempt spans are the audit trail

```
source = otel-v1-apm-span-*
| where name = 'tool.attempt'
| stats count() as attempts,
    max(`span.attributes.tool@retry@attempt`)
    as max_attempt
  by traceId, `span.attributes.tool@name`
| where max_attempt > 1
```

traceId	tool	attempts	max_attempt
a3f8...	rollback_mcp	2	2
9c12...	rollback_mcp	2	2
f04a...	rollback_mcp	2	2

... 104 rows returned — every one rollback_mcp
Group by traceId + tool only — the per-attempt retry reason stays on each span in the waterfall.



Mean retrieval latency: **~184 ms**. Every dashboard is green

Percentiles by `retrieval.document_bucket` expose the failure boundary the mean hides

Duration is `durationInNanos` — the native top-level span field. No custom attribute needed

```
source = otel-v1-apm-span-*
| where name = 'retrieval.search'
| eval duration_ms = durationInNanos / 1000000
| stats avg(duration_ms) as avg_ms,
        percentile(duration_ms, 95) as p95_ms,
        percentile(duration_ms, 99) as p99_ms
by `span.attributes.retrieval@document_bucket`
```

doc_bucket	avg_ms	p95_ms	p99_ms
0-10	54	71	72
11-50	113	140	144
51-100	213	264	269
100+	402	474	479

p95 474 ms at 100+ docs

~6.7× the 0–10 bucket · a boundary, not noise

FIX cap and paginate retrieval above the boundary — a workload limit, not the “make retrieval faster” an average would have sent you to do



Retrieved: **5 signal docs** (circuit-breaker errors, deploy ID, ticket, runbook) + **60 INFO filler docs** that flood the window

Generated answer: “*Check your service configuration.*”
Generic — no retrieved fact referenced

`context_usage_check` is a custom span — a **triage signal**, not a hallucination detector

0.250 → 0.062

evidence-overlap collapses 4.0× exactly at the overflow boundary

FIX rank and dedupe before assembly, drop routine heartbeat noise, alert when overlap falls while overflow is true

```
source = otel-v1-apm-span-*
| where name = 'context_usage_check'
| stats
  avg(`span.attributes.context@evidence_overlap_ratio`)
  as avg_overlap,
  avg(`span.attributes.context@final_context_tokens`)
  as avg_tokens, count() as checks
by `span.attributes.context@overflow_detected`
```

overflow	checks	avg_overlap	avg_tokens
false	949	0.250	78
true	51	0.062	798

“Sometimes wrong” is a mystery. “Overlap drops 4× past 500 tokens” is a fix.



The agent reported success. Did the platform recover?

Every one of the 500 incidents ended with the agent writing “remediation complete” to the ticket

So we asked the platform. Same cluster, same window: the entitlement-api error stream the diagnosis agent had been reading

104 incidents never stopped erroring — and they are exactly the 104 whose rollback silently ran twice

104 reported fixed.
104 still broken.

the retry was not a latency tax — it was the damage

```
source = platform-logs
| where level = 'ERROR' and phase = 'post_remediation'
| stats count() as errors_after by incident_id
| sort - errors_after
```

incident_id	errors_after
a3f8...	8
9c12...	8
f04a...	8

Why no agent-observability tool can run this

It needs the agent's traces AND the platform's own logs in one place. A separate LLM-observability product holds the first and never the second — so it can tell you what the agent did, never whether it worked.



We shipped the three fixes. Same 500 incidents, replayed.

- 1 idempotency key on the rollback client — a retry can no longer double-apply
- 2 retrieval capped at the boundary the percentiles exposed
- 3 rank and dedupe before context assembly

Same seed, same incident population, same queries. Only the platform changed

```
source = otel-v1-apm-span-*
| where name = 'incident.remediate'
| ... | stats sum(retried), sum(heavy), sum(lost)
by `span.attributes.agent@release`
```

release	retried	heavy	evidence lost
2026.09.1	104	95	51
2026.09.2	0	0	0

104 → 0 · 95 → 0 · 51 → 0

and the token bill fell with them: \$0.0028 → \$0.0009 per run

This is the part teams skip.
Finding the bug is half the loop. A deterministic replay of the same incident population is what turns “we think it is better” into a number you can put in a change record
Same index, same queries, no special build.



Everything in the repo:

- docker-compose — OpenSearch 3.7 · Dashboards · Data Prepper 2.16 · OTel Collector
- the instrumented agent — every span name and attribute
- queries/*.ppl — expected results in the comments
- docs — the case study · span schema · privacy & redaction
- CDK app — stoppable AWS copy (~\$2.40/mo parked)
- the slide deck you are watching

Deterministic by design

Seed 42 → you will see these exact numbers — on any OTel SDK version.

Point the same instrumentation at your own MCP servers — nothing else changes.

github.com/eilhamv/cfp-opensearch

Apache-2.0 licensed · no keys · no customer data

```
$ git clone github.com/eilhamv/cfp-opensearch
$ make demo
```

```
≈ 8 min · 500 incidents × 2 releases · 21,104 spans
  · 4 services
```

Then: Dashboards → Observability → Trace Analytics



The retry your logs called success.

The latency your averages called normal.

The evidence your agent retrieved — but never used.

All queryable. Same cluster. Same PPL.

Prompt logs tell you what the model said. Traces tell you why your agent behaved that way

Thank you — questions?



Your Agent Is a Distributed System: Debugging AI Failures
with OpenTelemetry and OpenSearch